

```

import pygame
import random

CELL      = 20
COLS      = 28
ROWS      = 28
GRID_W    = COLS * CELL
GRID_H    = ROWS * CELL
HUD_H     = 50
WIN_W     = GRID_W
WIN_H     = GRID_H + HUD_H
GRID_TOP  = HUD_H

UP        = ( 0, -1)
DOWN      = ( 0,  1)
LEFT      = (-1,  0)
RIGHT     = ( 1,  0)

BLACK     = ( 0,  0,  0)
WHITE     = (255, 255, 255)
DARK      = ( 20,  20,  20)
GRAY      = (100, 100, 100)
LGRAY     = (200, 200, 200)
GREEN     = ( 50, 200,  80)
DKGREEN   = ( 30, 120,  50)
RED       = (220,  50,  50)
DKRED     = (120,  10,  10)
YELLOW    = (240, 200,  0)
ORANGE    = (255, 140,  0)
CYAN      = ( 0, 210, 220)
PURPLE    = (160,  50, 220)
BLUE      = ( 50, 120, 220)
WALL_C    = ( 80,  80, 100)

FOOD_NORMAL = "normal"
FOOD_SILVER = "silver"
FOOD_GOLD   = "gold"
FOOD_POISON = "poison"

FOOD_COLORS = {
    FOOD_NORMAL: RED,
    FOOD_SILVER: LGRAY,
    FOOD_GOLD:   YELLOW,
    FOOD_POISON: DKRED,

```

```

}

FOOD_WEIGHTS = {
    FOOD_NORMAL: 1,
    FOOD_SILVER: 2,
    FOOD_GOLD: 5,
    FOOD_POISON: 0,
}

PU_SPEED = "speed_boost"
PU_SLOW = "slow_motion"
PU_SHIELD = "shield"
PU_COLORS = {PU_SPEED: ORANGE, PU_SLOW: CYAN, PU_SHIELD: PURPLE}
PU_LABELS = {PU_SPEED: "⚡", PU_SLOW: "🐢", PU_SHIELD: "🛡️"}
PU_DURATION_MS = {PU_SPEED: 5000, PU_SLOW: 5000, PU_SHIELD: None}
PU_FIELD_TIMEOUT_MS = 8000

def _cell(x, y):
    return (x, y)

class Snake:
    def __init__(self, color=(50, 200, 80)):
        cx, cy = COLS // 2, ROWS // 2
        self.body = [_cell(cx, cy), _cell(cx-1, cy), _cell(cx-2, cy)]
        self.dir = RIGHT
        self.next_dir = RIGHT
        self.color = color
        self.grew = False

        self.active_pu = None
        self.pu_end_ms = 0
        self.shielded = False

    def change_dir(self, new_dir):
        if (new_dir[0] * -1, new_dir[1] * -1) != self.dir:
            self.next_dir = new_dir

    def step(self):
        self.dir = self.next_dir
        head = self.body[0]
        new_head = ((head[0] + self.dir[0]) % COLS,
                    (head[1] + self.dir[1]) % ROWS)
        self.body.insert(0, new_head)
        if not self.grew:
            self.body.pop()
        self.grew = False
        return new_head

```

```

def grow(self):
    self.grew = True

def shorten(self, n=2):

    for _ in range(n):
        if len(self.body) > 1:
            self.body.pop()
    return len(self.body) > 1

def head(self):
    return self.body[0]

def collect_pu(self, pu_type: str):
    self.active_pu = pu_type
    now = pygame.time.get_ticks()
    dur = PU_DURATION_MS.get(pu_type)
    self.pu_end_ms = (now + dur) if dur else 0
    if pu_type == PU_SHIELD:
        self.shielded = True

def update_pu(self):
    if self.active_pu and self.active_pu != PU_SHIELD:
        if pygame.time.get_ticks() >= self.pu_end_ms:
            self.active_pu = None

def speed_modifier(self) -> float:
    if self.active_pu == PU_SPEED: return 0.6
    if self.active_pu == PU_SLOW: return 1.6
    return 1.0

def on_collision(self) -> bool:

    if self.shielded:
        self.shielded = False
        self.active_pu = None
        return False
    return True

def draw(self, surface, show_grid=False):
    for i, (cx, cy) in enumerate(self.body):
        r = pygame.Rect(cx*CELL, GRID_TOP + cy*CELL, CELL, CELL)
        col = self.color if i > 0 else DKGREEN
        pygame.draw.rect(surface, col, r, border_radius=4)
        pygame.draw.rect(surface, DARK, r, 1, border_radius=4)
    if show_grid:
        pass

```

```

class FoodItem:
    def __init__(self, pos, ftype, timeout_ms=0):
        self.pos      = pos
        self.type      = ftype
        self.spawn_ms  = pygame.time.get_ticks()
        self.timeout   = timeout_ms

    def expired(self) -> bool:
        if self.timeout == 0:
            return False
        return pygame.time.get_ticks() - self.spawn_ms > self.timeout

    def draw(self, surface):
        cx, cy = self.pos
        r = pygame.Rect(cx*CELL+2, GRID_TOP + cy*CELL+2, CELL-4, CELL-4)
        pygame.draw.ellipse(surface, FOOD_COLORS[self.type], r)
        if self.type == FOOD_POISON:
            pygame.draw.ellipse(surface, BLACK, r, 2)

class PowerUpItem:
    def __init__(self, pos, pu_type):
        self.pos      = pos
        self.type      = pu_type
        self.spawn_ms  = pygame.time.get_ticks()

    def expired(self) -> bool:
        return pygame.time.get_ticks() - self.spawn_ms > PU_FIELD_TIMEOUT_MS

    def draw(self, surface, font):
        cx, cy = self.pos
        r = pygame.Rect(cx*CELL, GRID_TOP + cy*CELL, CELL, CELL)
        pygame.draw.rect(surface, PU_COLORS[self.type], r, border_radius=5)
        pygame.draw.rect(surface, WHITE, r, 1, border_radius=5)

class GameWorld:
    BASE_SPEED_MS = 180

    def __init__(self, settings: dict, personal_best: int = 0):
        self.settings      = settings
        self.personal_best = personal_best

        color = tuple(settings.get("snake_color", [50, 200, 80]))
        self.snake      = Snake(color=color)
        self.foods      : list[FoodItem]      = []
        self.powerup     : PowerUpItem | None = None
        self.obstacles  : set[tuple]          = set()

```

```

self.score          = 0
self.level          = 1
self.food_eaten     = 0
self.food_per_level = 5

self._spawn_food()

self.font_hud = pygame.font.SysFont("consolas", 16, bold=True)
self.font_sm  = pygame.font.SysFont("consolas", 13)

self._last_step = pygame.time.get_ticks()
self._pu_timer  = pygame.time.get_ticks()
self._pu_interval = 8000

def _free_cells(self) -> list[tuple]:
    occupied = set(self.snake.body) | self.obstacles
    if self.powerup:
        occupied.add(self.powerup.pos)
    for f in self.foods:
        occupied.add(f.pos)
    return [(x, y) for x in range(COLS) for y in range(ROWS)
            if (x, y) not in occupied]

def _rand_free(self):
    free = self._free_cells()
    return random.choice(free) if free else None

def _spawn_food(self):
    pos = self._rand_free()
    if not pos:
        return
    roll = random.random()
    if roll < 0.12:
        ftype = FOOD_GOLD
        timeout = 4000
    elif roll < 0.30:
        ftype = FOOD_SILVER
        timeout = 6000
    elif roll < 0.42:
        ftype = FOOD_POISON
        timeout = 5000
    else:
        ftype = FOOD_NORMAL
        timeout = 0
    self.foods.append(FoodItem(pos, ftype, timeout))

```

```

def _spawn_obstacles(self):
    if self.level < 3:
        return
    count      = 3 + (self.level - 3) * 2
    head       = self.snake.head()
    safe_zone = {(head[0]+dx, head[1]+dy)
                  for dx in range(-2, 3) for dy in range(-2, 3)}
    self.obstacles.clear()
    free = [c for c in self._free_cells() if c not in safe_zone]
    random.shuffle(free)
    for cell in free[:count]:
        self.obstacles.add(cell)

def _try_spawn_pu(self):
    now = pygame.time.get_ticks()
    if self.powerup is None and now - self._pu_timer > self._pu_interval:
        pos = self._rand_free()
        if pos:
            pt = random.choice([PU_SPEED, PU_SLOW, PU_SHIELD])
            self.powerup = PowerUpItem(pos, pt)
            self._pu_timer = now

def _level_up(self):
    self.level      += 1
    self.food_eaten = 0
    self._spawn_obstacles()

def _step_ms(self) -> int:
    base = max(60, self.BASE_SPEED_MS - (self.level - 1) * 12)
    return int(base * self.snake.speed_modifier())

def update(self) -> str:

    now = pygame.time.get_ticks()
    self.snake.update_pu()
    self._try_spawn_pu()

    self.foods = [f for f in self.foods if not f.expired()]
    if self.powerup and self.powerup.expired():
        self.powerup = None

    if now - self._last_step < self._step_ms():
        return "alive"
    self._last_step = now

    head = self.snake.step()

```

```

    if head in self.snake.body[1:]:
        if self.snake.on_collision():
            return "dead"

    if head in self.obstacles:
        if self.snake.on_collision():
            return "dead"

    eaten = None
    for f in self.foods:
        if f.pos == head:
            eaten = f
            break

    if eaten:
        self.foods.remove(eaten)
        if eaten.type == FOOD_POISON:
            if not self.snake.shorten(2):
                return "dead"
        else:
            self.snake.grow()
            pts = FOOD_WEIGHTS[eaten.type] * self.level
            self.score += pts
            self.food_eaten += 1
            if self.food_eaten >= self.food_per_level:
                self._level_up()
            self._spawn_food()

    if self.powerup and self.powerup.pos == head:
        self.snake.collect_pu(self.powerup.type)
        self.powerup = None

    return "alive"

def draw(self, surface):
    show_grid = self.settings.get("grid", False)

    surface.fill(DARK)

    if show_grid:
        for x in range(COLS):
            for y in range(ROWS):
                r = pygame.Rect(x*CELL, GRID_TOP + y*CELL, CELL, CELL)
                pygame.draw.rect(surface, (35, 35, 35), r, 1)

    for (cx, cy) in self.obstacles:
        r = pygame.Rect(cx*CELL, GRID_TOP + cy*CELL, CELL, CELL)
        pygame.draw.rect(surface, WALL_C, r)

```

```

        pygame.draw.rect(surface, GRAY, r, 1)

    for f in self.foods:
        f.draw(surface)

    if self.powerup:
        self.powerup.draw(surface, self.font_sm)

    self.snake.draw(surface, show_grid)

    self._draw_hud(surface)

def _draw_hud(self, surface):
    pygame.draw.rect(surface, (30, 30, 30), (0, 0, WIN_W, HUD_H))

    def txt(t, x, y, col=WHITE):
        lbl = self.font_hud.render(t, True, col)
        surface.blit(lbl, (x, y))

    txt(f"Score: {self.score}", 6, 6)
    txt(f"Lvl: {self.level}", 6, 26)
    txt(f"PB: {self.personal_best}", 160, 6, YELLOW)

    pu = self.snake.active_pu
    if pu:
        remaining = ""
        if pu != PU_SHIELD:
            ms_left = max(0, self.snake.pu_end_ms - pygame.time.get_ticks())
            remaining = f" {ms_left//1000+1}s"
        col = PU_COLORS[pu]
        label = {PU_SPEED: "BOOST", PU_SLOW: "SLOW", PU_SHIELD: "SHIELD"}[pu]
        txt(f"{label}{remaining}", WIN_W//2 - 30, 6, col)

    txt(f"Food: {self.food_eaten}/{self.food_per_level}", WIN_W - 120, 6)
    txt(f"Len: {len(self.snake.body)}", WIN_W - 120, 26)

```